

LINUX PLUMBERS CONFERENCE 2023

SYN Proxy at Scale with BPF

"Can I modify the internal TCP socket sequence number using eBPF?"

Kuniyuki Iwashima

Kernel Development Engineer
Amazon Web Services

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Agenda

SYN Cookie in Linux

SYN Proxy at Scale

SYN Proxy at Scale with BPF

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

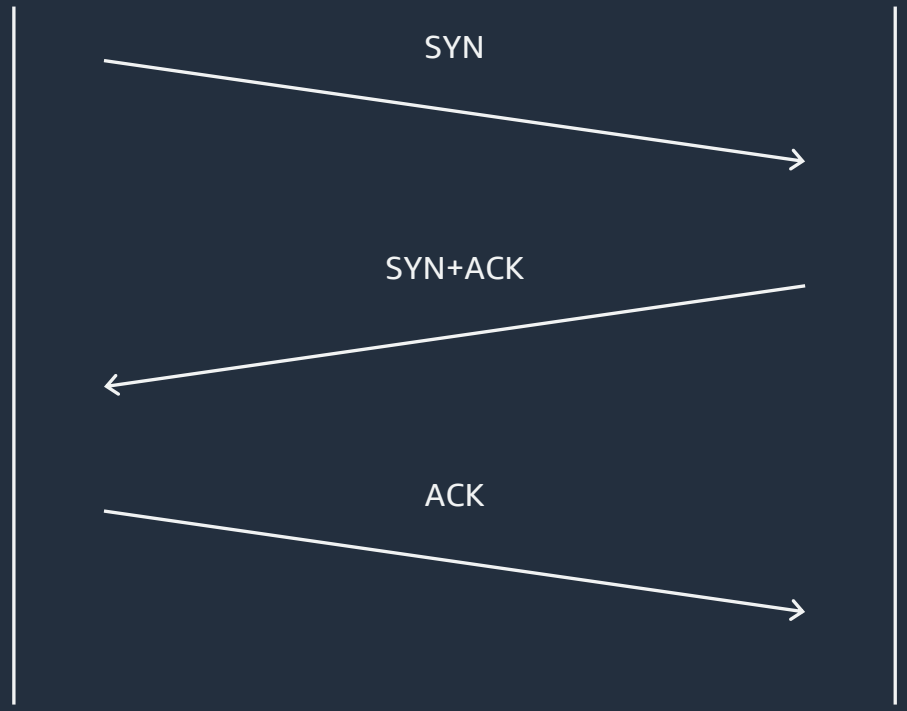
SYN Cookie in Linux



© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

SYN Cookie







Encode client info into
SYN+ACK



Stateless

Decode client info from
ACK

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

SYN Cookie - Initial Sequence Number





SYN
SEQ: X, ACK: 0

SYN+ACK
SEQ: C, ACK: X+1

ACK
SEQ: X+1, ACK: C+1

Acknowledge ISN

Encode client info into
ISN

Stateless

Decode client info from
ISN (ACK# - 1)

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

SYN Cookie - Timestamps Option





SYN
TSval: t1, TSecr: 0

SYN+ACK
TSval: **t2**, TSecr: t1

ACK
TSval: t3, TSecr: **t2**

Encode client info into **TSval**

Stateless

Echo back **TSval**

Decode client info from **TSecr**

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

What's encoded in ISN ?

net/ipv4/syncookies.c

```
static __u32 secure_tcp_syn_cookie(__be32 saddr, __be32 daddr, __be16 sport,
                                   __be16 dport, __u32 sseq, __u32 data)
{
        /* Compute the secure sequence number.
         * The output should be:
         *   HASH(sec1,saddr,sport,daddr,dport,sec1) + sseq + (count * 2^24)
         *      + (HASH(sec2,saddr,sport,daddr,dport,count,sec2) % 2^24).
         * Where sseq is their sequence number and count increases every
         * minute by 1.
         * As an extra hack, we add a small "data" value that encodes the
         * MSS into the second hash value.
         */
        u32 count = tcp_cookie_time();

        return (cookie_hash(saddr, daddr, sport, dport, 0, 0) + sseq + (count << COOKIEBITS) +
                ((cookie_hash(saddr, daddr, sport, dport, count, 1) + data) & COOKIEMASK));
}
```

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

4-tuple

net/ipv4/syncookies.c

```
static __u32 secure_tcp_syn_cookie(__be32 saddr, __be32 daddr, __be16 sport,
                                   __be16 dport, __u32 sseq, __u32 data)
{
        /* Compute the secure sequence number.
         * The output should be:
         *   HASH(sec1,saddr,sport,daddr,dport,sec1) + sseq + (count * 2^24)
         *      + (HASH(sec2,saddr,sport,daddr,dport,count,sec2) % 2^24).
         * Where sseq is their sequence number and count increases every
         * minute by 1.
         * As an extra hack, we add a small "data" value that encodes the
         * MSS into the second hash value.
         */
        u32 count = tcp_cookie_time();

        return (cookie_hash(saddr, daddr, sport, dport, 0, 0) + sseq + (count << COOKIEBITS) +
                ((cookie_hash(saddr, daddr, sport, dport, count, 1) + data) & COOKIEMASK));
}
```

aws

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Client's ISN

net/ipv4/syncookies.c

```
static __u32 secure_tcp_syn_cookie(__be32 saddr, __be32 daddr, __be16 sport,
                                   __be16 dport, __u32 sseq, __u32 data)
{
        /* Compute the secure sequence number.
         * The output should be:
         *   HASH(sec1,saddr,sport,daddr,dport,sec1) + sseq + (count * 2^24)
         *      + (HASH(sec2,saddr,sport,daddr,dport,count,sec2) % 2^24).
         * Where sseq is their sequence number and count increases every
         * minute by 1.
         * As an extra hack, we add a small "data" value that encodes the
         * MSS into the second hash value.
         */
        u32 count = tcp_cookie_time();

        return (cookie_hash(saddr, daddr, sport, dport, 0, 0) + sseq + (count << COOKIEBITS) +
                ((cookie_hash(saddr, daddr, sport, dport, count, 1) + data) & COOKIEMASK));
}
```

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

MSS (Maximum Segment Size)

net/ipv4/syncookies.c

```
static __u32 secure_tcp_syn_cookie(__be32 saddr, __be32 daddr, __be16 sport,
                                   __be16 dport, __u32 sseq, __u32 data)
{
        /* Compute the secure sequence number.
         * The output should be:
         *   HASH(sec1,saddr,sport,daddr,dport,sec1) + sseq + (count * 2^24)
         *      + (HASH(sec2,saddr,sport,daddr,dport,count,sec2) % 2^24).
         * Where sseq is their sequence number and count increases every
         * minute by 1.
         * As an extra hack, we add a small "data" value that encodes the
         * MSS into the second hash value.
         */
        u32 count = tcp_cookie_time();

        return (cookie_hash(saddr, daddr, sport, dport, 0, 0) + sseq + (count << COOKIEBITS) +
                ((cookie_hash(saddr, daddr, sport, dport, count, 1) + data) & COOKIEMASK));
}
```

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

MSS (Maximum Segment Size)

net/ipv4/syncookies.c

```
static __u16 const msstab[] = {
        536, 1300, 1440, 1460,
};

u32 __cookie_v4_init_sequence(const struct iphdr *iph, const struct tcphdr *th, u16 *mssp)
{
        int mssind;
        const __u16 mss = *mssp;

        for (mssind = ARRAY_SIZE(msstab) - 1; mssind ; mssind--)
                if (mss >= msstab[mssind])
                        break;

        *mssp = msstab[mssind];

        return secure_tcp_syn_cookie(iph->saddr, iph->daddr, th->source, th->dest,
                                     ntohl(th->seq), mssind);
}
```

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

System Timestamp

net/ipv4/syncookies.c

```
static __u32 secure_tcp_syn_cookie(__be32 saddr, __be32 daddr, __be16 sport,
                                   __be16 dport, __u32 sseq, __u32 data)
{
        /* Compute the secure sequence number.
         * The output should be:
         *   HASH(sec1,saddr,sport,daddr,dport,sec1) + sseq + (count * 2^24)
         *      + (HASH(sec2,saddr,sport,daddr,dport,count,sec2) % 2^24).
         * Where sseq is their sequence number and count increases every
         * minute by 1.
         * As an extra hack, we add a small "data" value that encodes the
         * MSS into the second hash value.
         */
        u32 count = tcp_cookie_time();

        return (cookie_hash(saddr, daddr, sport, dport, 0, 0) + sseq + (count << COOKIEBITS) +
                ((cookie_hash(saddr, daddr, sport, dport, count, 1) + data) & COOKIEMASK));
}
```

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

System Timestamp

include/net/tcp.h

```
/* Syncookies use a monotonic timer which increments every 60 seconds.
 * This counter is used both as a hash input and partially encoded into
 * the cookie value.  A cookie is only validated further if the delta
 * between the current counter value and the encoded one is less than this,
 * i.e. a sent cookie is valid only at most for 2*60 seconds (or less if
 * the counter advances immediately after a cookie is generated).
 */
#define MAX_SYNCOOKIE_AGE 2
#define TCP_SYNCOOKIE_PERIOD (60 * HZ)
#define TCP_SYNCOOKIE_VALID (MAX_SYNCOOKIE_AGE * TCP_SYNCOOKIE_PERIOD)

static inline u32 tcp_cookie_time(void)
{
        u64 val = get_jiffies_64();
        do_div(val, TCP_SYNCOOKIE_PERIOD);
        return val;
}
```

aws
© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Secrets

net/ipv4/syncookies.c

```
static __u32 secure_tcp_syn_cookie(__be32 saddr, __be32 daddr, __be16 sport,
                                   __be16 dport, __u32 sseq, __u32 data)
{
        /* Compute the secure sequence number.
         * The output should be:
         *   HASH(sec1,saddr,sport,daddr,dport,sec1) + sseq + (count * 2^24)
         *      + (HASH(sec2,saddr,sport,daddr,dport,count,sec2) % 2^24).
         * Where sseq is their sequence number and count increases every
         * minute by 1.
         * As an extra hack, we add a small "data" value that encodes the
         * MSS into the second hash value.
         */
        u32 count = tcp_cookie_time();

        return (cookie_hash(saddr, daddr, sport, dport, 0, 0) + sseq + (count << COOKIEBITS) +
                ((cookie_hash(saddr, daddr, sport, dport, count, 1) + data) & COOKIEMASK));
}
```

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Secrets

net/ipv4/syncookies.c

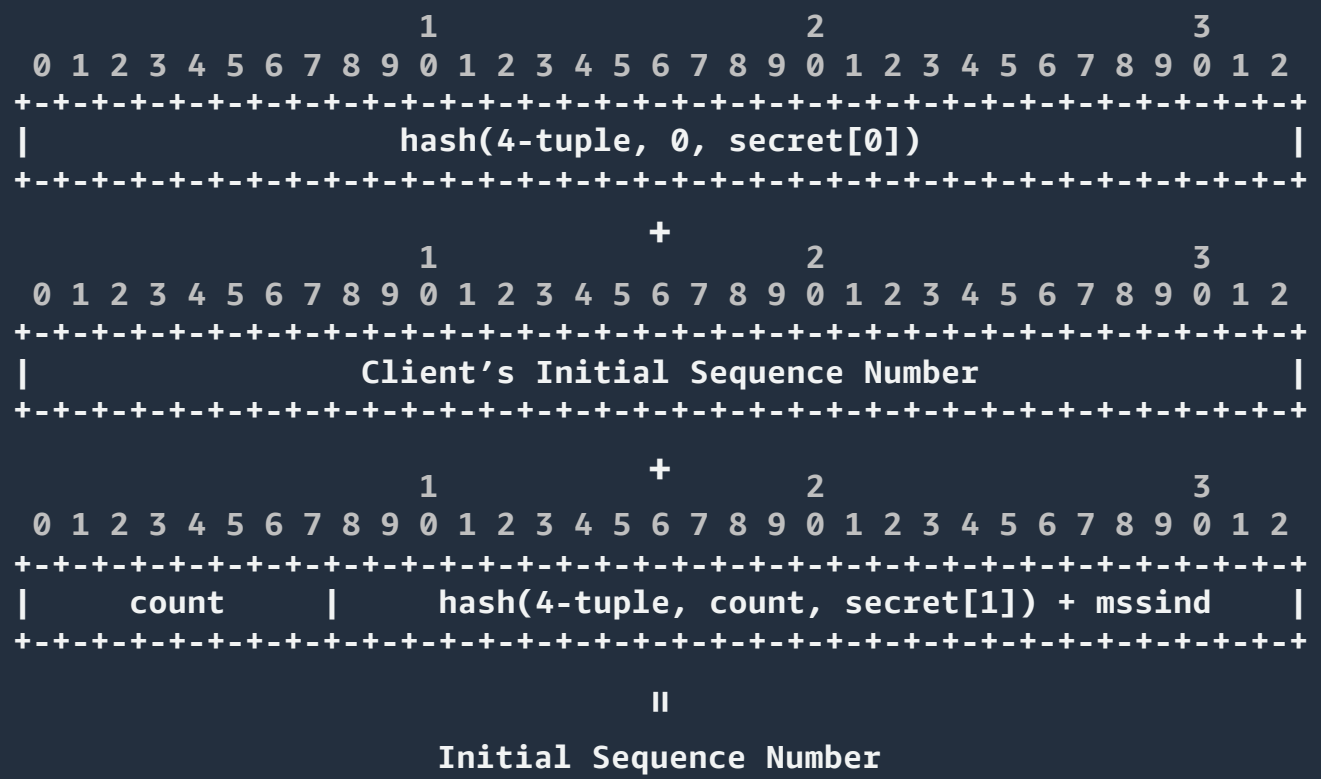
```
static siphash_aligned_key_t syncookie_secret[2];


static u32 cookie_hash(__be32 saddr, __be32 daddr, __be16 sport, __be16 dport,
                       u32 count, int c)
{
        net_get_random_once(syncookie_secret, sizeof(syncookie_secret));

        return siphash_4u32((__force u32)saddr, (__force u32)daddr,
                            (__force u32)sport << 16 | (__force u32)dport,
                            count, &syncookie_secret[c]);
}
```

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

What’s encoded in ISN ?



- 4-tuple
- Client’s ISN
- MSS approximation
  - IPv4 : 536, 1300, 1440, 1460
  - IPv6 : 1220, 1420, 1440, 8940
- System Timestamp (**count**)
  - Valid for 2 minutes
  - **`jiffies / (60 * HZ)`**
- Secrets
  - **`net_get_random_once()`**

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

What’s encoded in TSval ?

net/ipv4/syncookies.c

```
#define TSBITS 6

u64 cookie_init_timestamp(struct request_sock *req, u64 now)
{
        const struct inet_request_sock *ireq = inet_rsk(req);
        u64 ts, ts_now = tcp_ns_to_ts(false, now);
        u32 options = 0;

        options = ireq->wscale_ok ? ireq->snd_wscale : TS_OPT_WSCALE_MASK;
        if (ireq->sack_ok)
                options |= TS_OPT_SACK;
        if (ireq->ecn_ok)
                options |= TS_OPT_ECN;

        ts = (ts_now >> TSBITS) << TSBITS;
        ts |= options;
        if (ts > ts_now)
                ts -= (1UL << TSBITS);
...
```

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

TSval with the lower 6 bits cleared

net/ipv4/syncookies.c

```
#define TS_OPT_WSCALE_MASK 0xf
#define TS_OPT_SACK BIT(4)
#define TS_OPT_ECN BIT(5)
#define TSBITS 6

u64 cookie_init_timestamp(struct request_sock *req, u64 now)
{
        const struct inet_request_sock *ireq = inet_rsk(req);
        u64 ts, ts_now = tcp_ns_to_ts(false, now);
        u32 options = 0;

        options = ireq->wscale_ok ? ireq->snd_wscale : TS_OPT_WSCALE_MASK;
        if (ireq->sack_ok)
                options |= TS_OPT_SACK;
        if (ireq->ecn_ok)
                options |= TS_OPT_ECN;

        ts = (ts_now >> TSBITS) << TSBITS;
        ts |= options;
...
```

aws

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

WScale, SACK, and ECN

net/ipv4/syncookies.c

```
#define TS_OPT_WSCALE_MASK 0xf
#define TS_OPT_SACK BIT(4)
#define TS_OPT_ECN BIT(5)
#define TSBITS 6

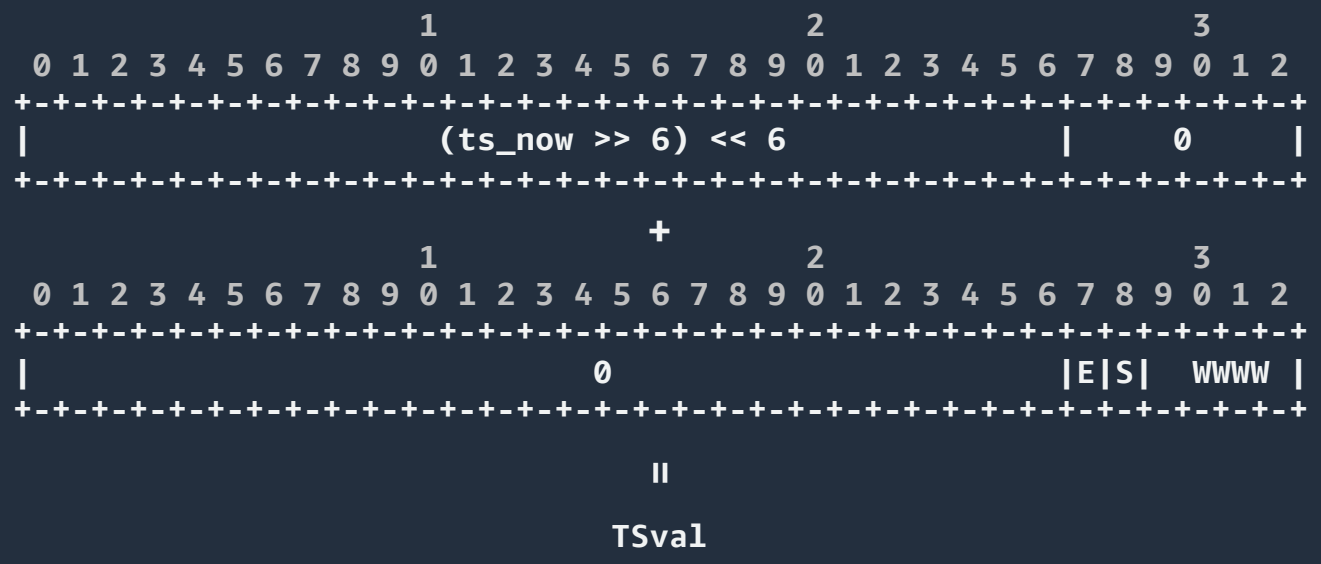
u64 cookie_init_timestamp(struct request_sock *req, u64 now)
{
        const struct inet_request_sock *ireq = inet_rsk(req);
        u64 ts, ts_now = tcp_ns_to_ts(false, now);
        u32 options = 0;

        options = ireq->wscale_ok ? ireq->snd_wscale : TS_OPT_WSCALE_MASK;
        if (ireq->sack_ok)
                options |= TS_OPT_SACK;
        if (ireq->ecn_ok)
                options |= TS_OPT_ECN;

        ts = (ts_now >> TSBITS) << TSBITS;
        ts |= options;
...
```

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

What’s encoded in TSval ?



- Window Scaling
  - 0 ~ 14
  - 15 : No WScale in SYN
- SACK Permitted
- ECN Supported

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

What's encoded in SYN Cookie

Initial Sequence Number

- 4-tuple
- Client's ISN
- MSS approximation
  - IPv4: 536, 1300, 1440, 1460
  - IPv6: 1220, 1420, 1440, 8940
- System Timestamp
  - Valid for 2 minutes
  - **`jiffies / (60 * HZ)`**
- Secrets
  - **`net_get_random_once()`**

Timestamps Option

- Window Scaling
  - 0 ~ 14
  - 15 : No WScale in SYN
- SACK Permitted
- ECN Supported

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

What's encoded in SYN Cookie

Initial Sequence Number

- 4-tuple
- Client's ISN
- MSS approximation
  - IPv4: 536, 1300, 1440, 1460
  - IPv6: 1220, 1420, 1440, 8940
- System Timestamp
  - Valid for 2 minutes
  - `jiffies / (60 * HZ)`
- Secrets
  - `net_get_random_once()`

Timestamps Option

- Window Scaling
  - 0 ~ 14
  - 15 : No WScale in SYN
- SACK Permitted
- ECN Supported

Drawbacks

- Precise MSS could be lost
- WScale, SACK, and ECN info could be lost without TS

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

What's encoded in SYN Cookie

Initial Sequence Number

- 4-tuple
- Client's ISN
- MSS approximation
  - IPv4: 536, 1300, 1440, 1460
  - IPv6: 1220, 1420, 1440, 8940
- System Timestamp
  - Valid for 2 minutes
  - `jiffies / (60 * HZ)`
- Secrets
  - `net_get_random_once()`

Timestamps Option

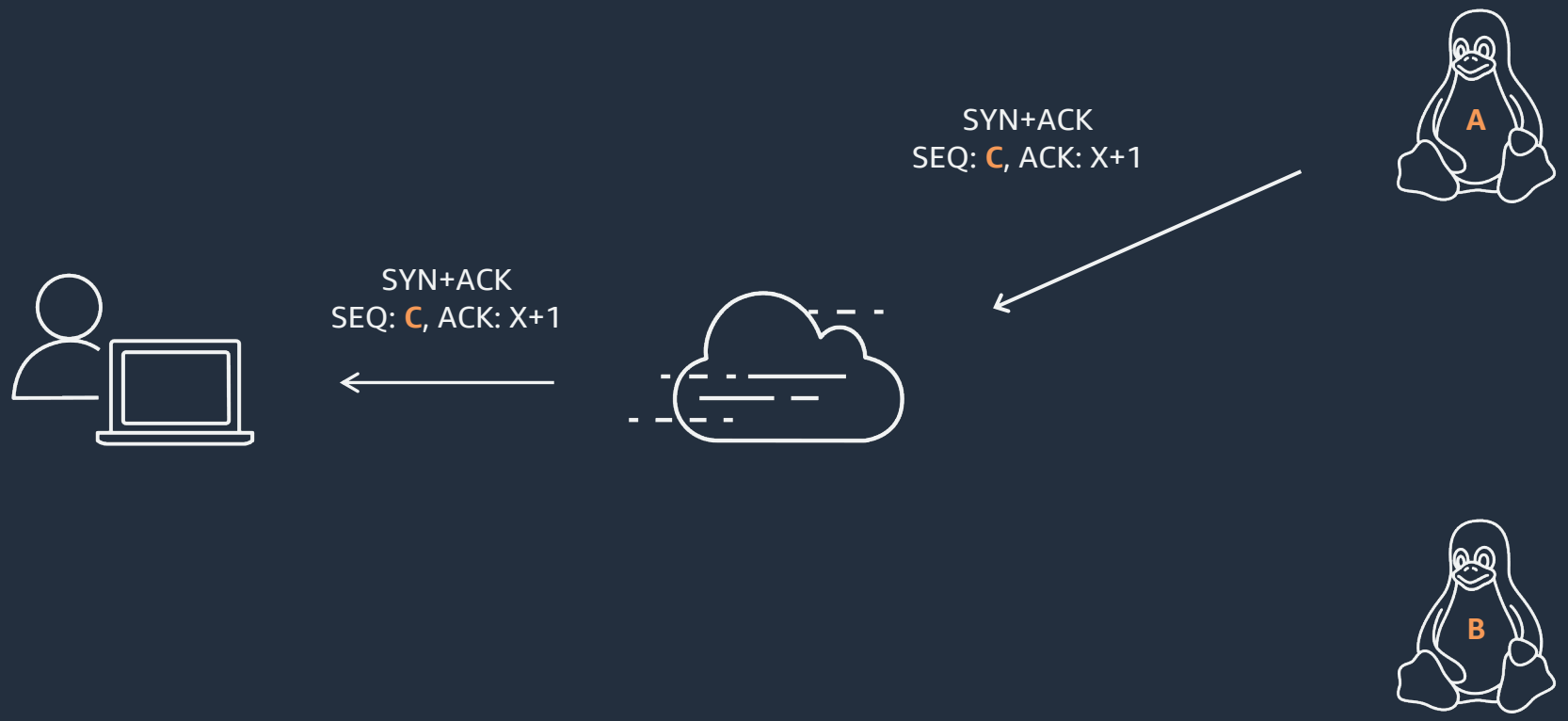
- Window Scaling
  - 0 ~ 14
  - 15 : No WScale in SYN
- SACK Permitted
- ECN Supported

Drawbacks

- Precise MSS could be lost
- WScale, SACK, and ECN info could be lost without TS
- **Host-specific bits are included**

aws
© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

If ACK is misrouted...



© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

If ACK is misrouted...



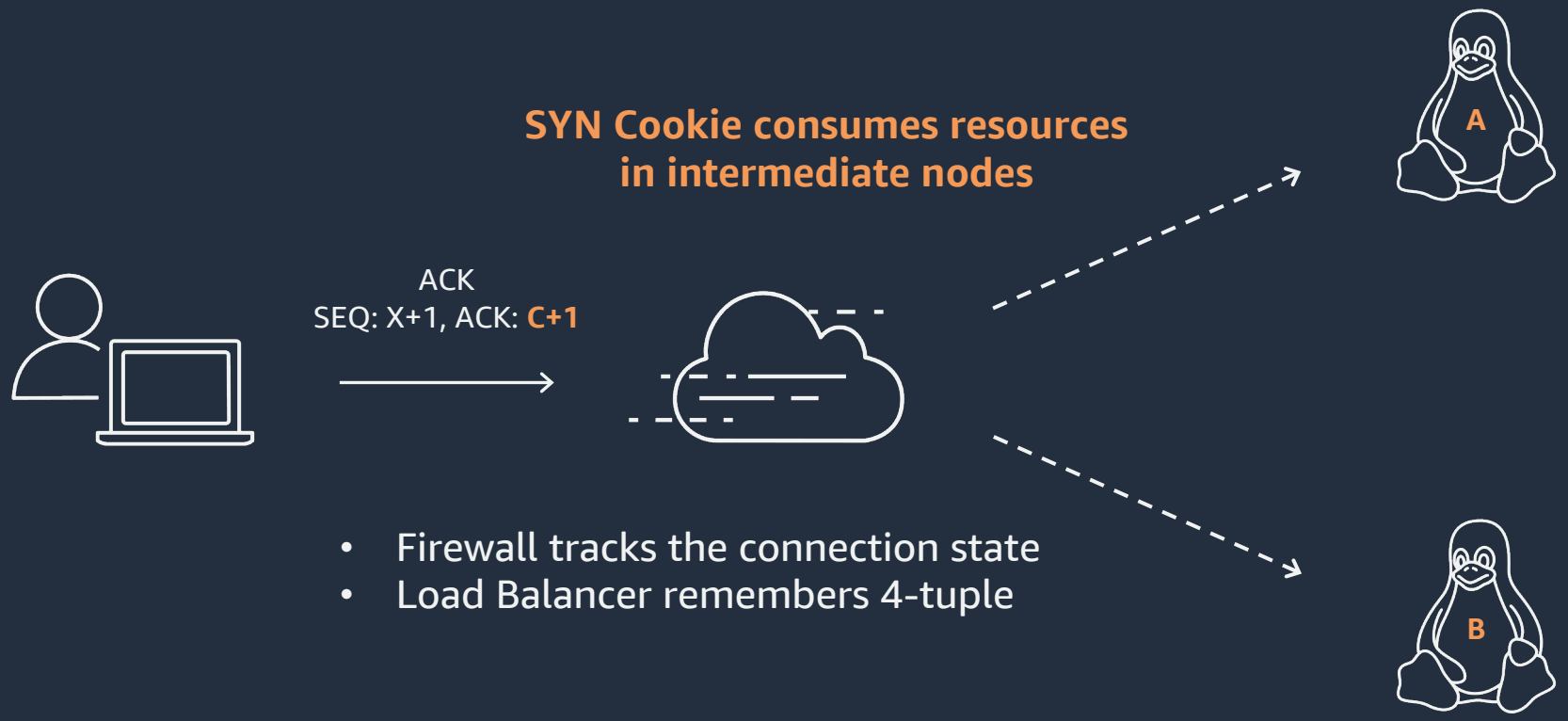
© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

If ACK is misrouted...



© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

SYN Cookie is *NOT stateless in network*



- Firewall tracks the connection state
- Load Balancer remembers 4-tuple

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

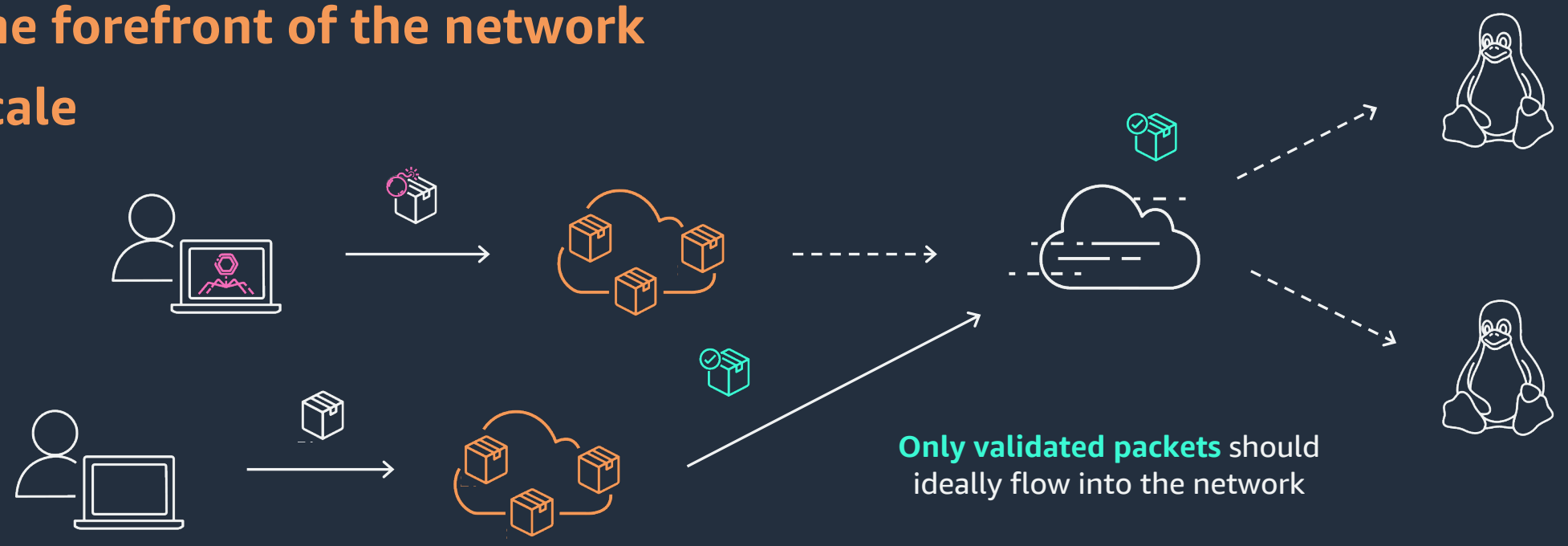
SYN Proxy at Scale



© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

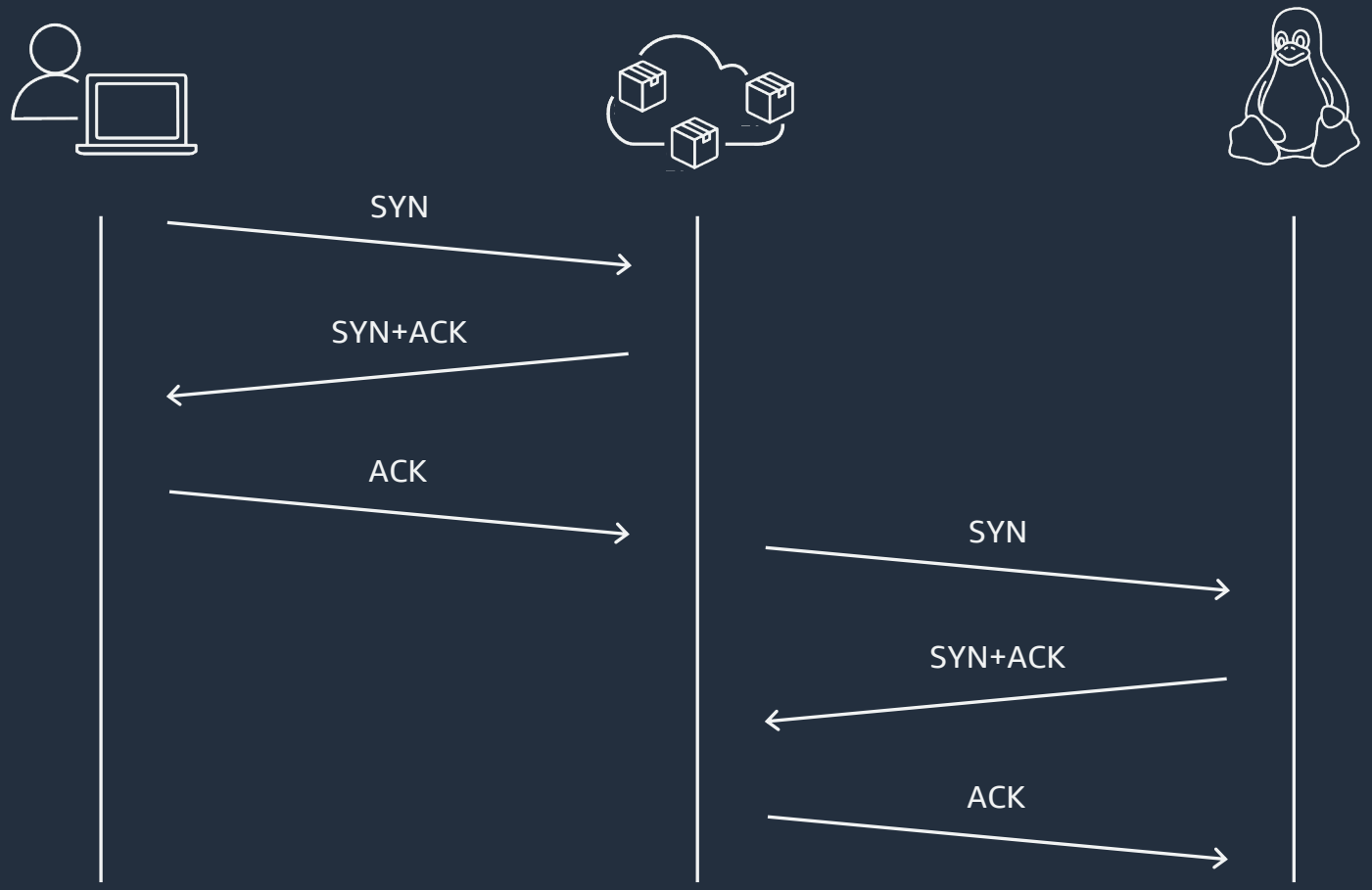
One of our services

- Serve multiple terabytes per second in large regions
- Need SYN flood protection
  - **At the forefront of the network**
  - **At scale**



© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

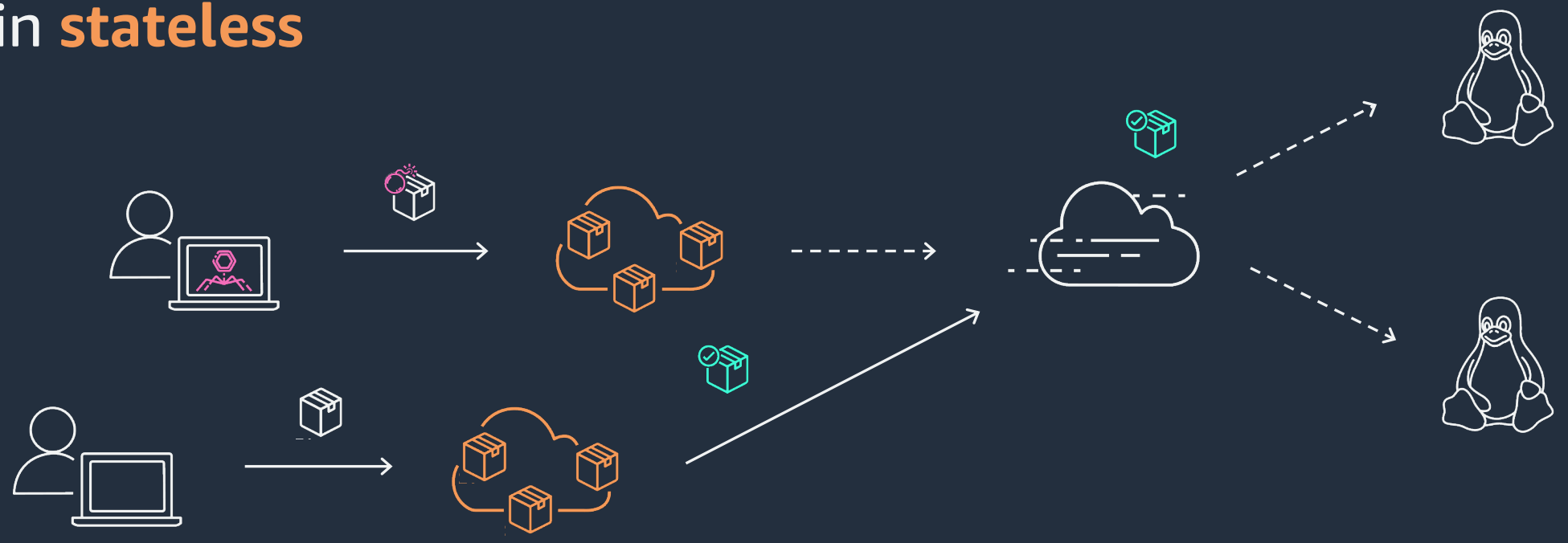
SYN Proxy



© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Tenets of SYN Proxy

- Reduce SYN flood to a managable level
- Never sacrifice each connection quality
- Remain **stateless**



© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Conventional SYN Proxy is stateful

- Netfilter **`SYNPROXY`** module
- BPF helpers
  - XDP : **`bpf_raw_(gen|check)_syncookie_ipv[46]`**
  - TC : **`bpf_(gen|check)_syncookie`**

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Conventional SYN Proxy is stateful

- Netfilter **`SYNPROXY`** module
- BPF helpers
  - XDP : **`bpf_raw_(gen|check)_syncookie_ipv[46]`**
  - TC : **`bpf_(gen|check)_syncookie`**

1. Reuse SYN Cookie infra

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

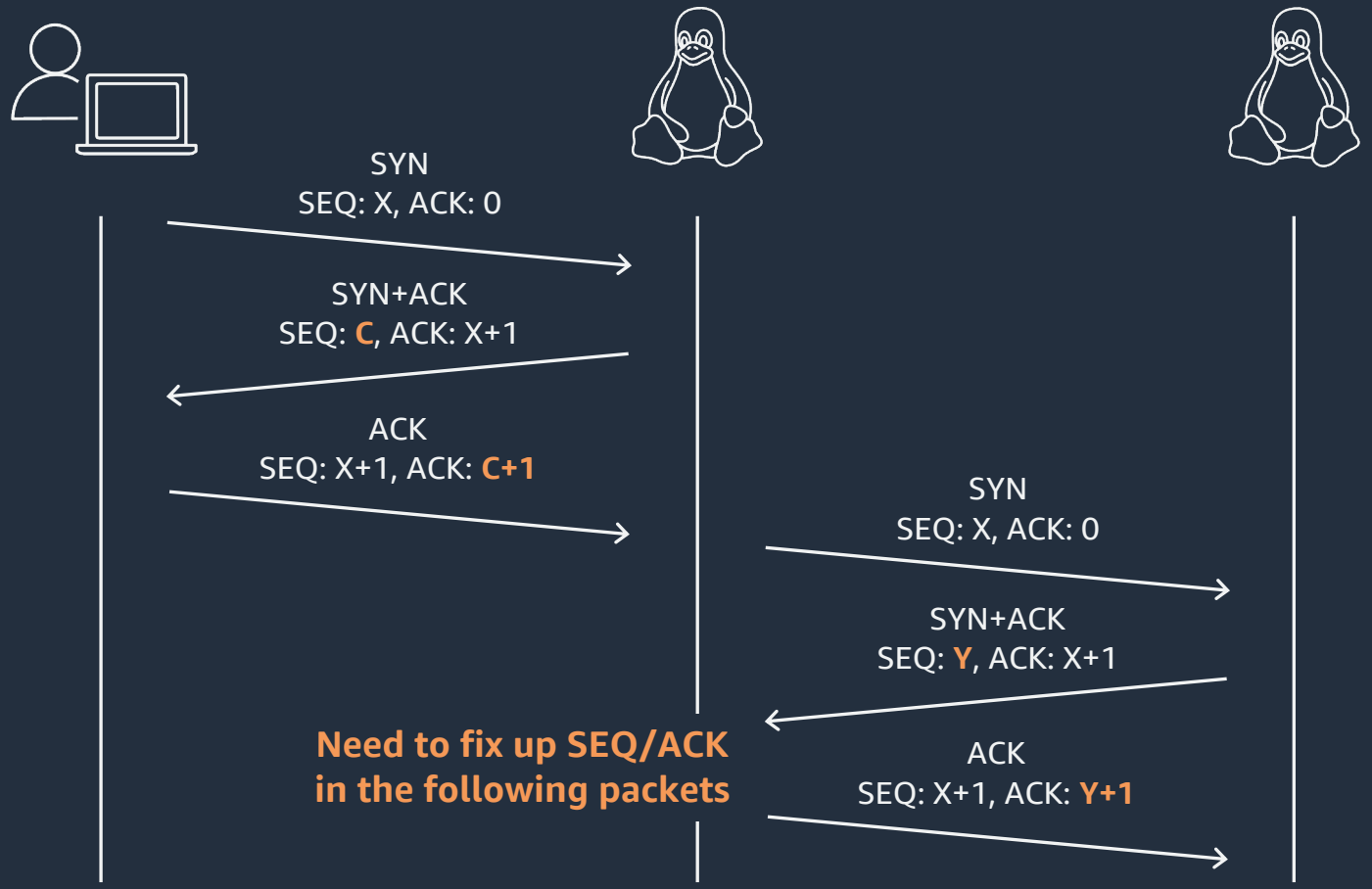
Conventional SYN Proxy is stateful

- Netfilter **`SYNPROXY`** module
- BPF helpers
  - XDP : **`bpf_raw_(gen|check)_syncookie_ipv[46]`**
  - TC : **`bpf_(gen|check)_syncookie`**

1. Reuse SYN Cookie infra
2. Need to manage ISN mappings

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Conventional SYN Proxy is stateful



© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Conventional SYN Proxy is stateful

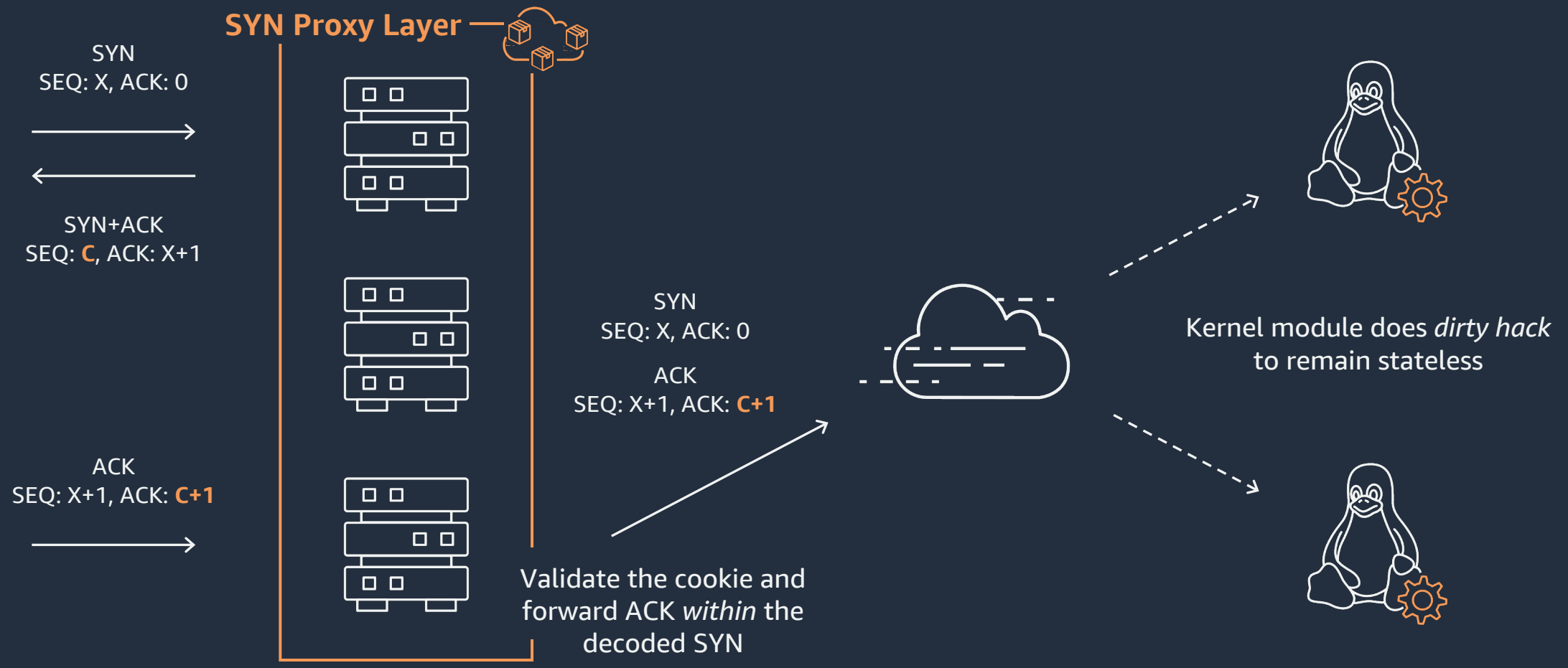
- Netfilter **`SYNPROXY`** module
- BPF helpers
  - XDP : **`bpf_raw_(gen|check)_syncookie_ipv[46]`**
  - tc : **`bpf_(gen|check)_syncookie`**

1. Reuse exisiting SYN Cookie infrastructure
2. Need to manage ISN mappings

→ **Hard to replace and scale**

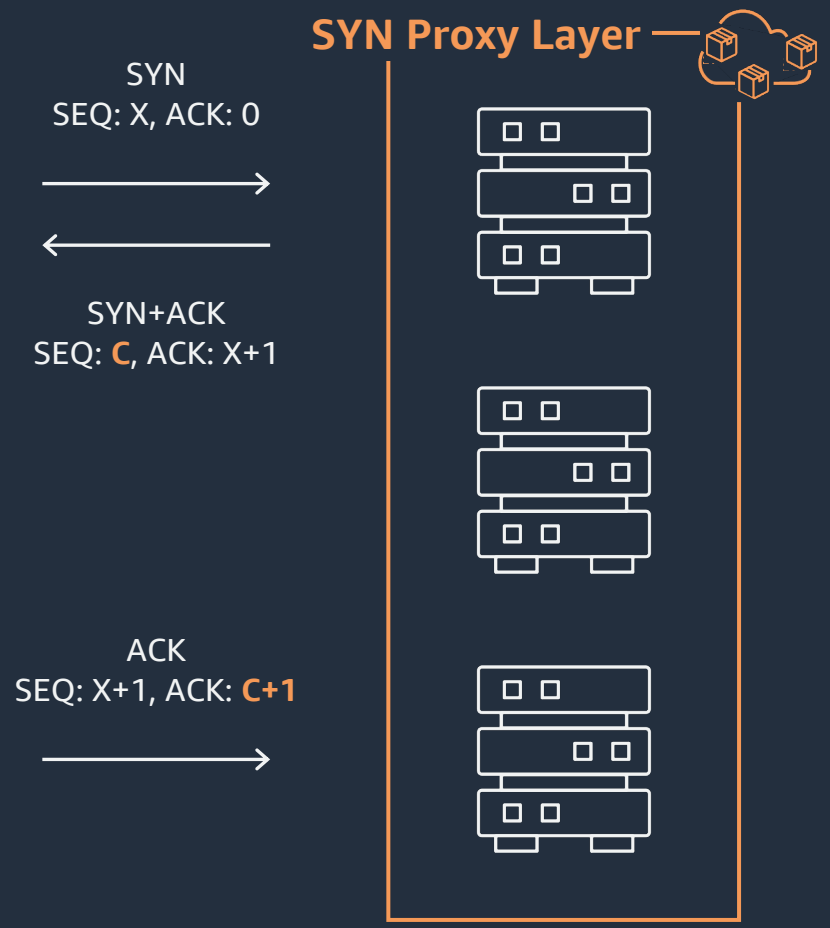
© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

SYN Proxy at Scale



© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

What's encoded in ISN ?




```
                      1                   2   3
 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 .. 0 1
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+  +-+-+
|S| WWWW  | MMM | HHHHHHHHHHHHHHHHHHHHHHH ..    |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+  +-+-+
S: SACK Permitted  W: Window Scale  M: MSS  H: Hash
```

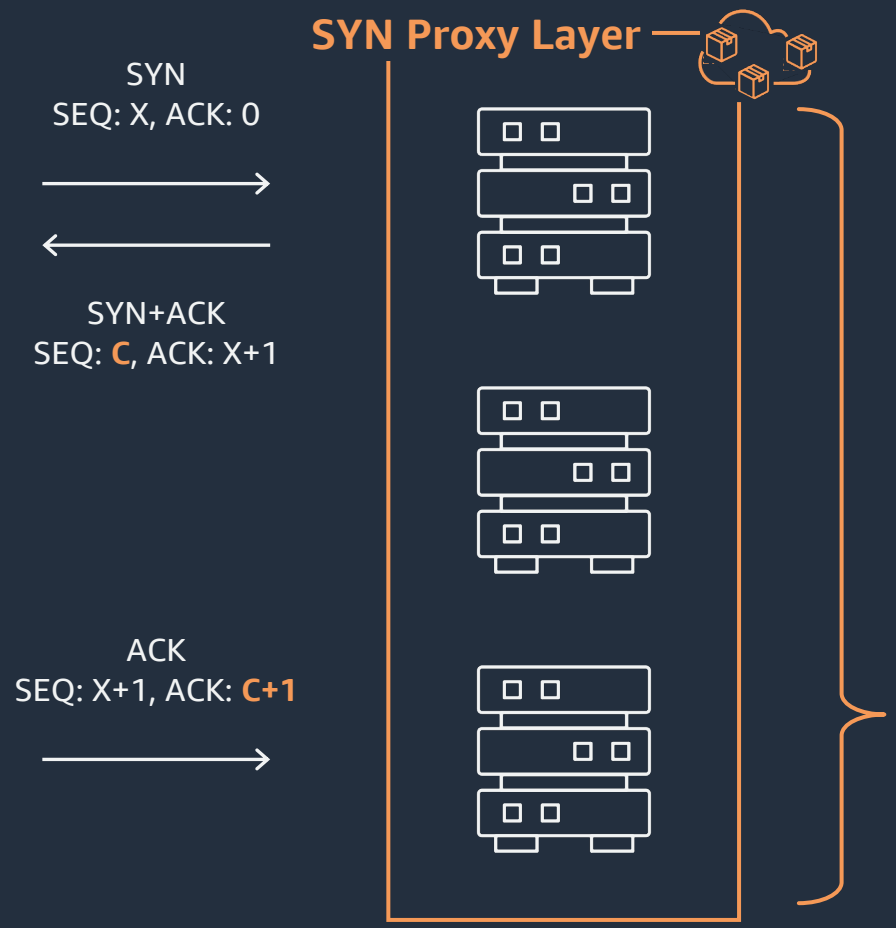
ISN includes:

- SACK Permitted
- WScale

  → **Keep connection quality without TS**
- MSS approximation
- Hash

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

What's encoded in ISN ?



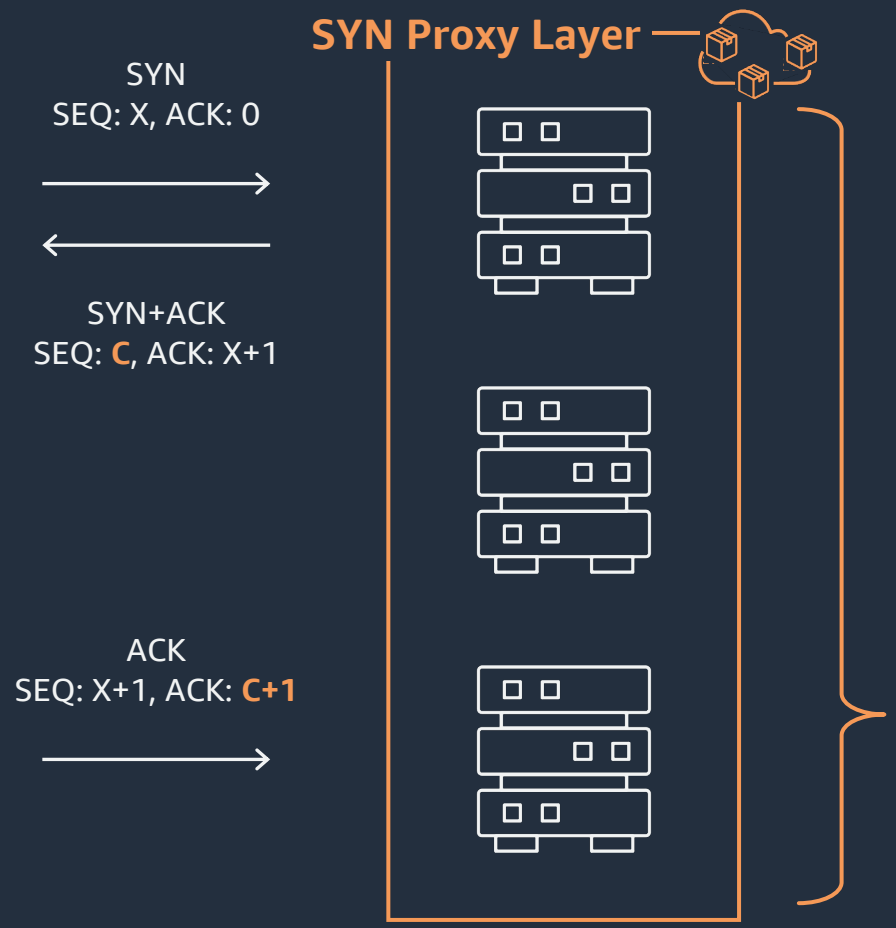

```
                      1                   2   3
 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 .. 0 1
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+  +-+-+
|S| WWWW  | MMM | HHHHHHHHHHHHHHHHHHHHHHHH ..    |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+  +-+-+
S: SACK Permitted  W: Window Scale  M: MSS  H: Hash
```

Hash includes:

- 4-tuple
- Client's ISN
- Client's TCP Options
- **Rolling Salt :** periodically-updated random number

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

What's encoded in ISN ?




```
                      1                   2   3
 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 .. 0 1
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+  +-+-+
|S| WWWW  | MMM | HHHHHHHHHHHHHHHHHHHHHHHH ..    |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+  +-+-+
S: SACK Permitted  W: Window Scale  M: MSS  H: Hash
```

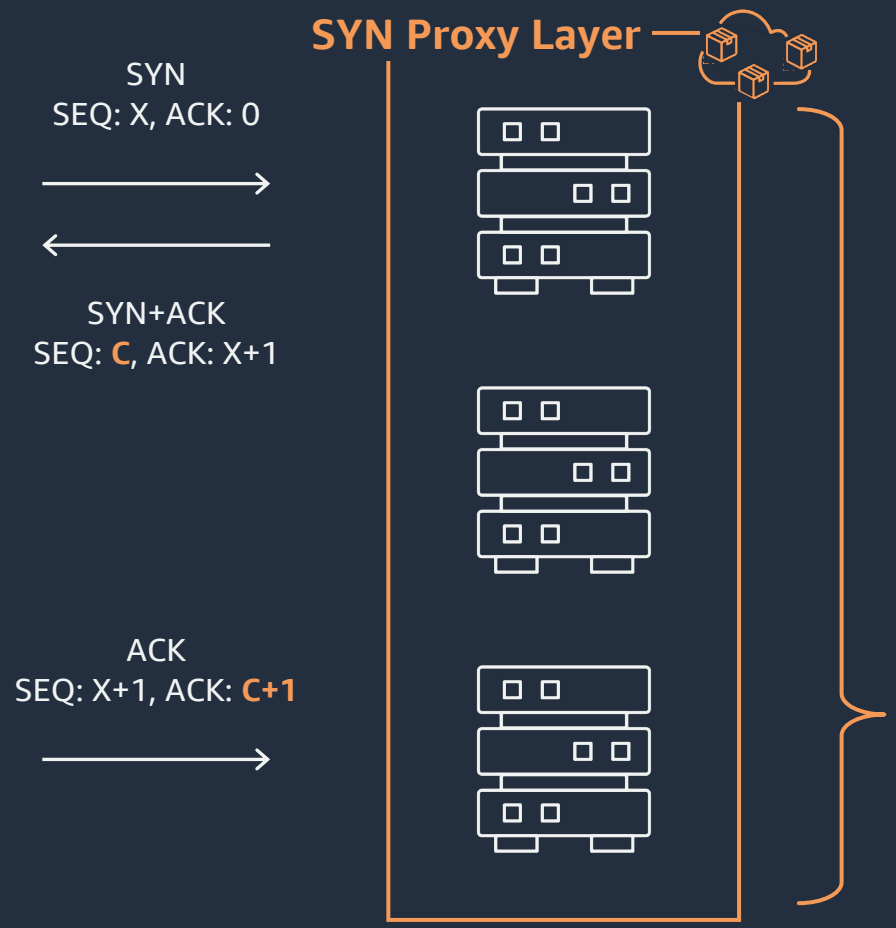
Hash includes:

- 4-tuple
- Client's ISN
- Client's TCP Options
- **Rolling Salt :** periodically-updated random number

  → **All node share the same secret**

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

What's encoded in ISN ?




```
                      1                   2   3
 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 .. 0 1
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+  +-+-+
|S| WWWW  | MMM | HHHHHHHHHHHHHHHHHHHHHHHH ..    |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+  +-+-+
S: SACK Permitted  W: Window Scale  M: MSS  H: Hash
```

Hash includes:

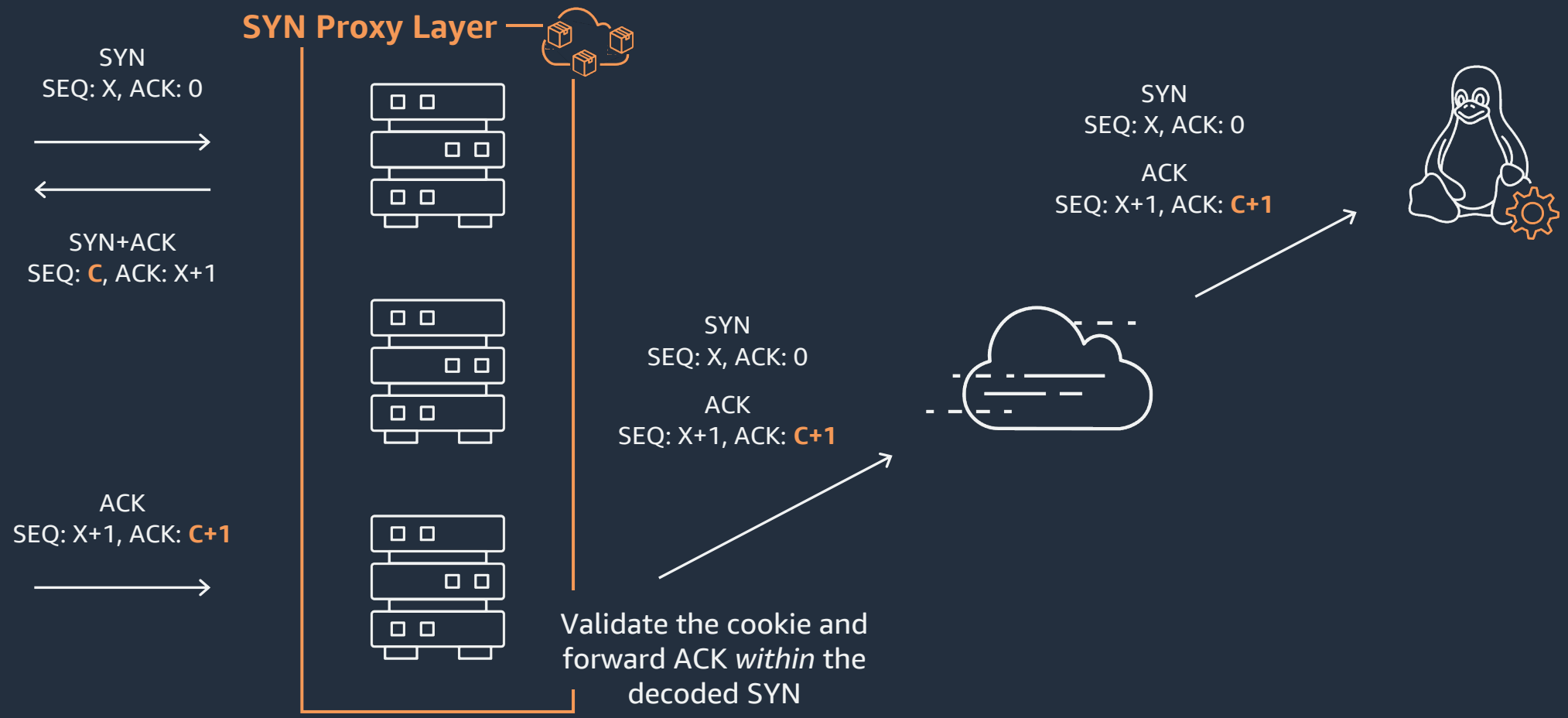
- 4-tuple
- Client's ISN
- Client's TCP Options
- **Rolling Salt :** periodically-updated random number

  → **All node share the same secret**

  → **Any node can generate/validate SYN Cookie statelessly**

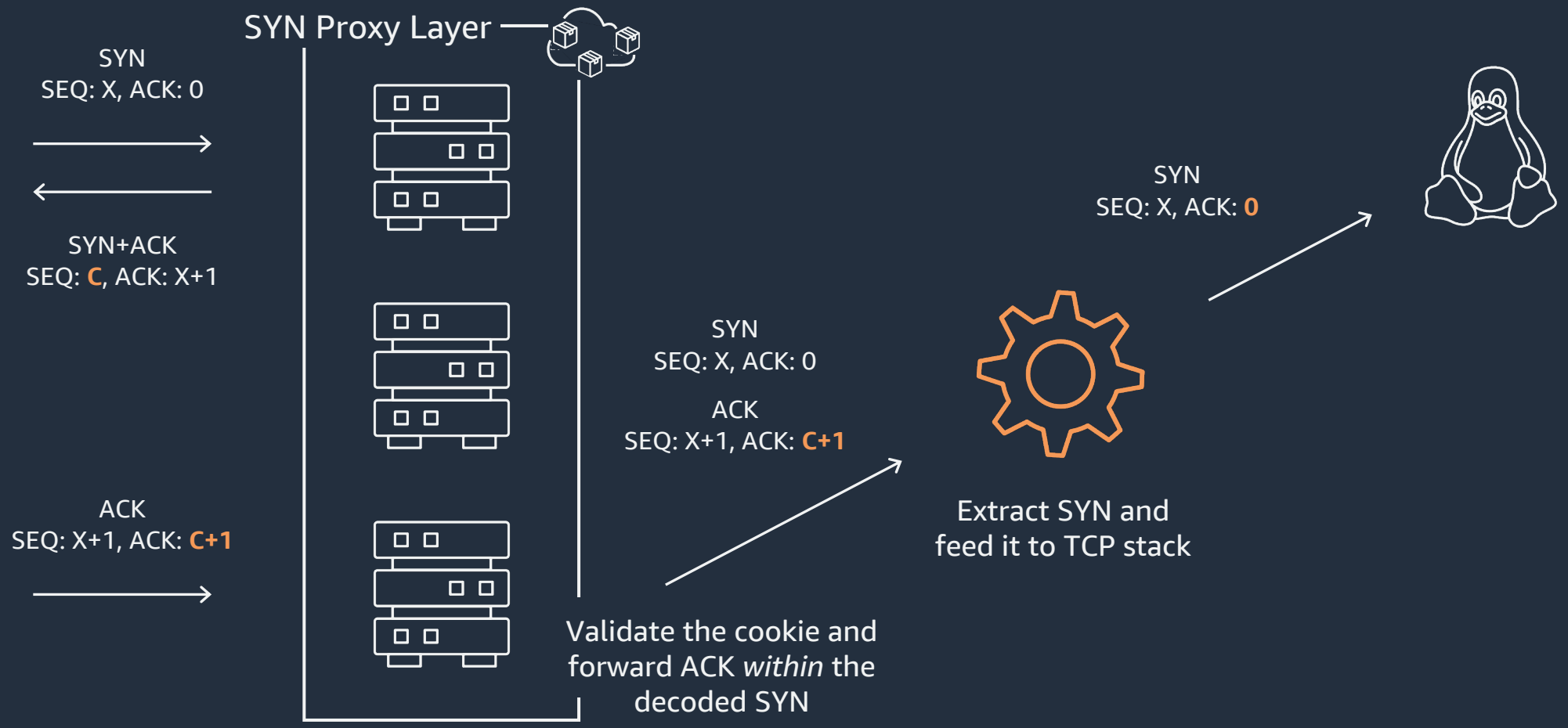
© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

How to remain stateless ?



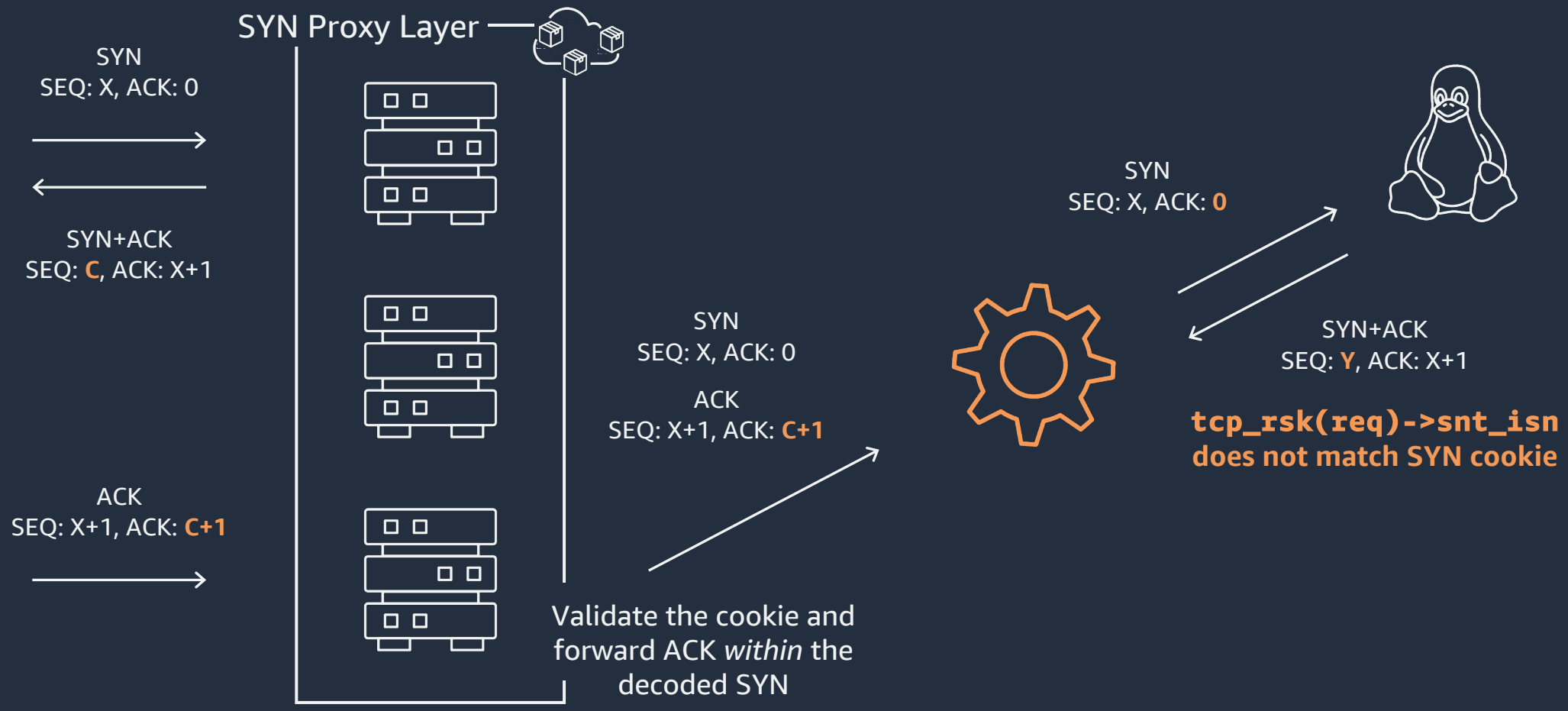
© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

How to remain stateless ?



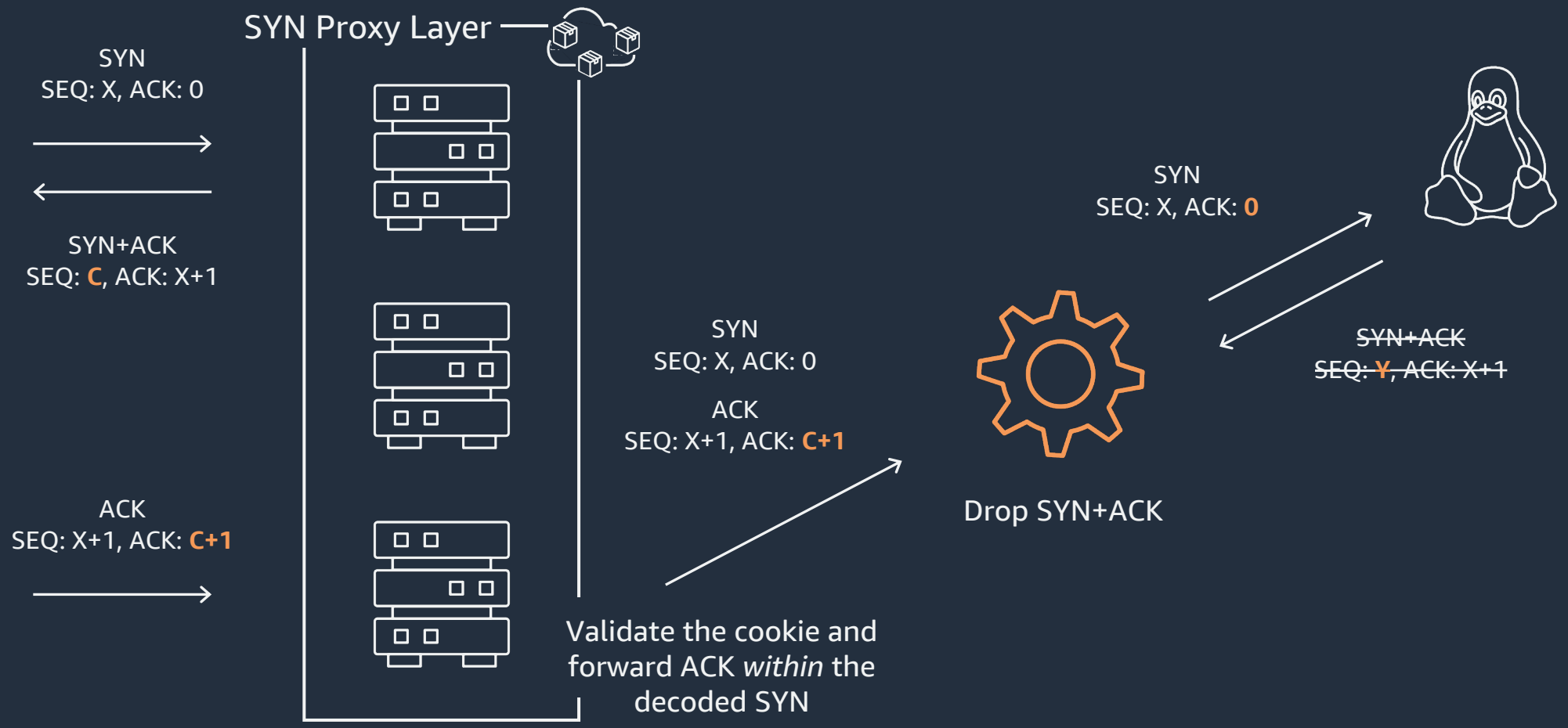
© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

How to remain stateless ?



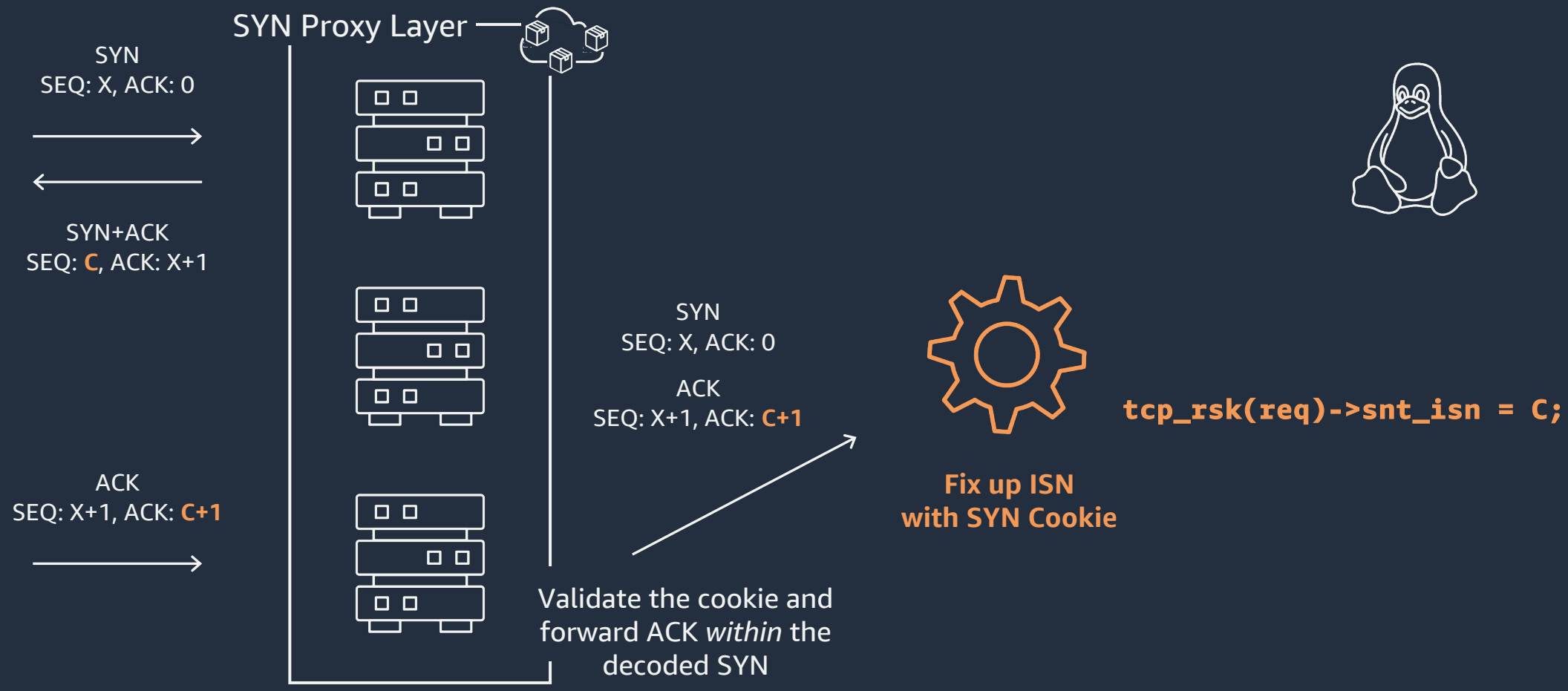
© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

How to remain stateless ?



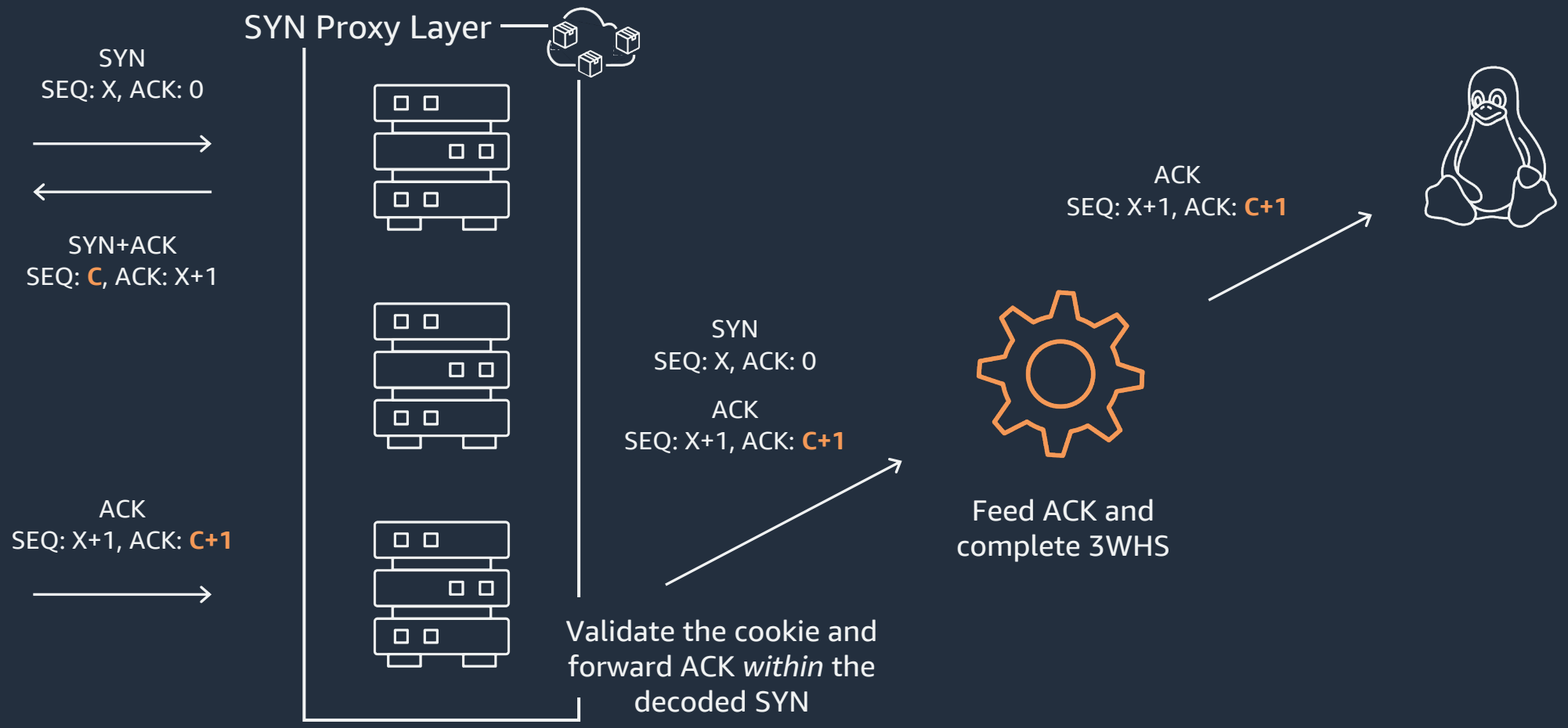
© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

How to remain stateless ?



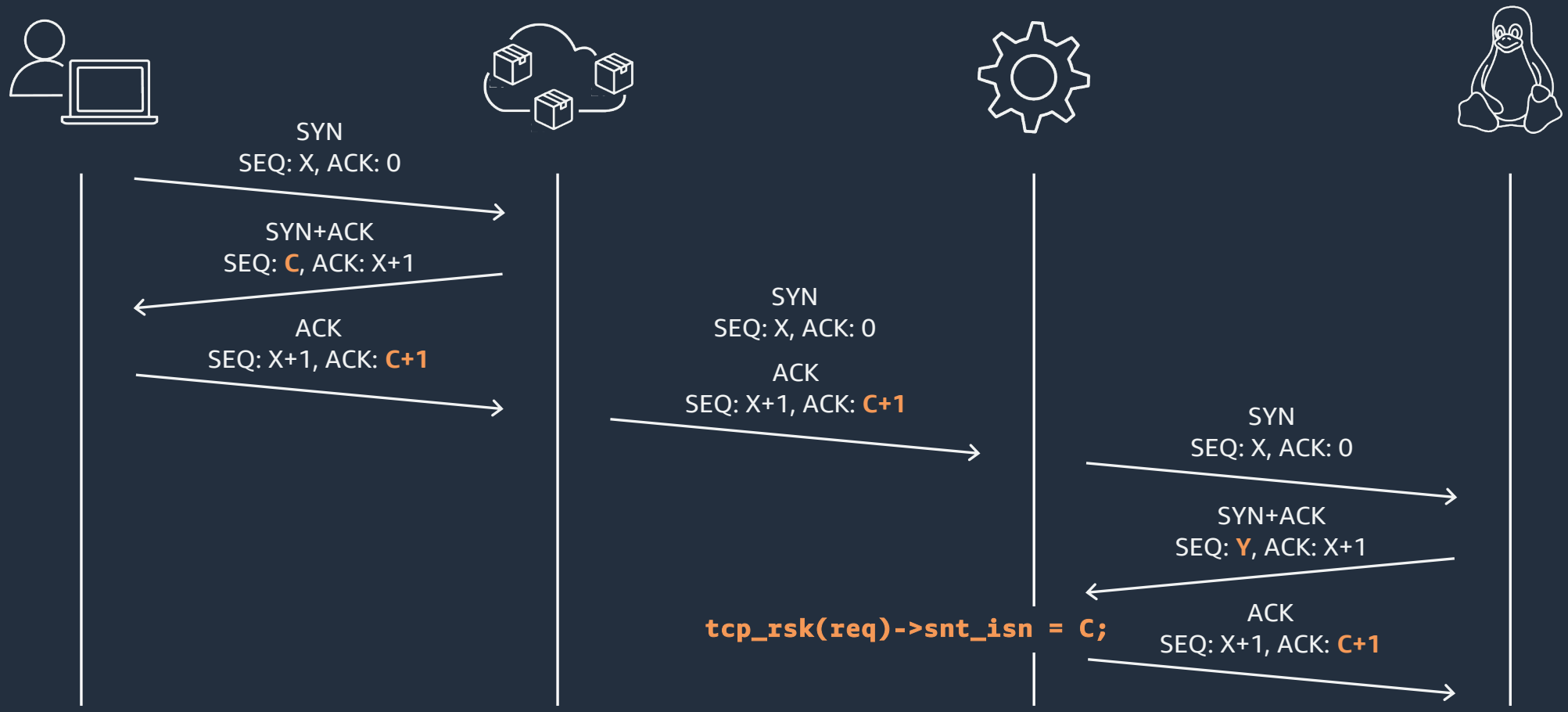
© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

How to remain stateless ?



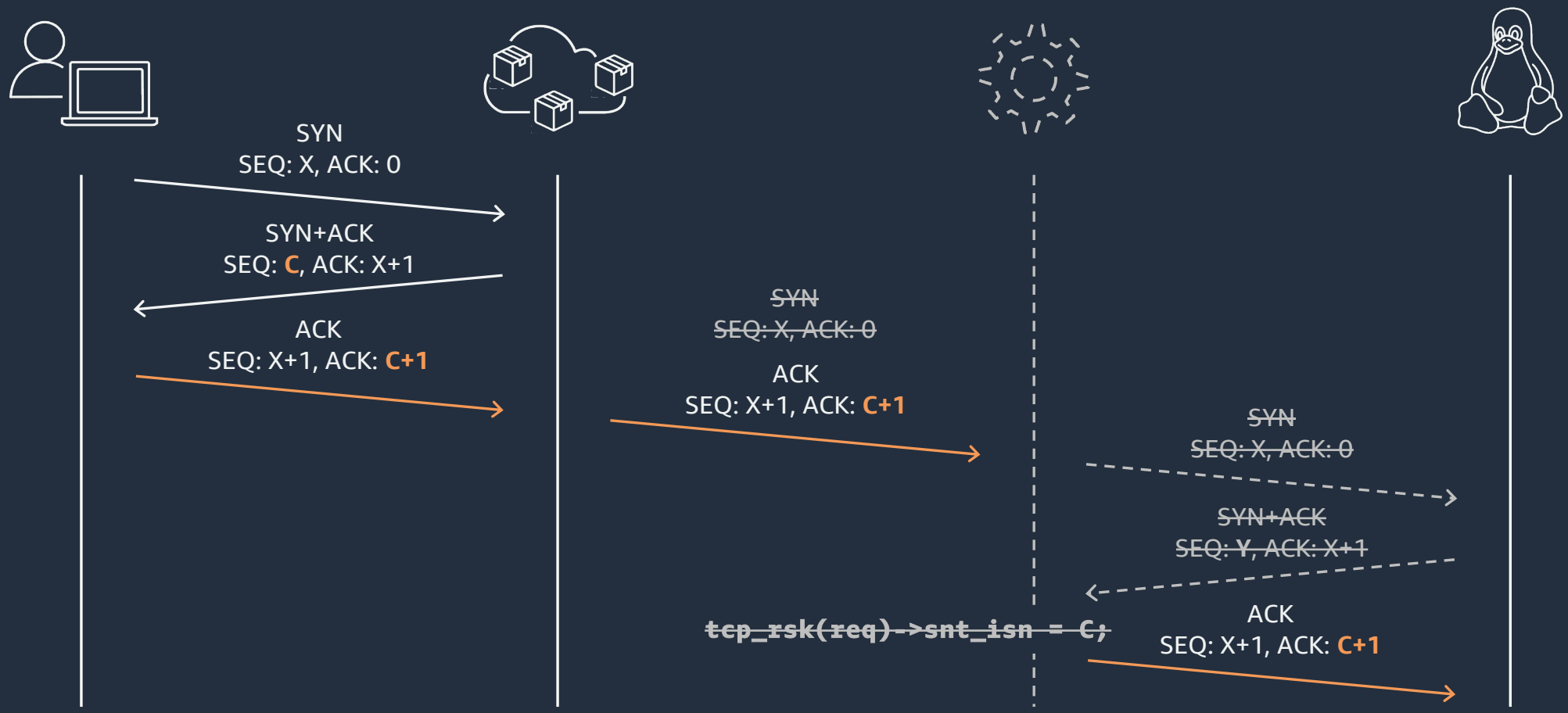
© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Stateless SYN Proxy



© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

If we could validate custom SYN Cookie



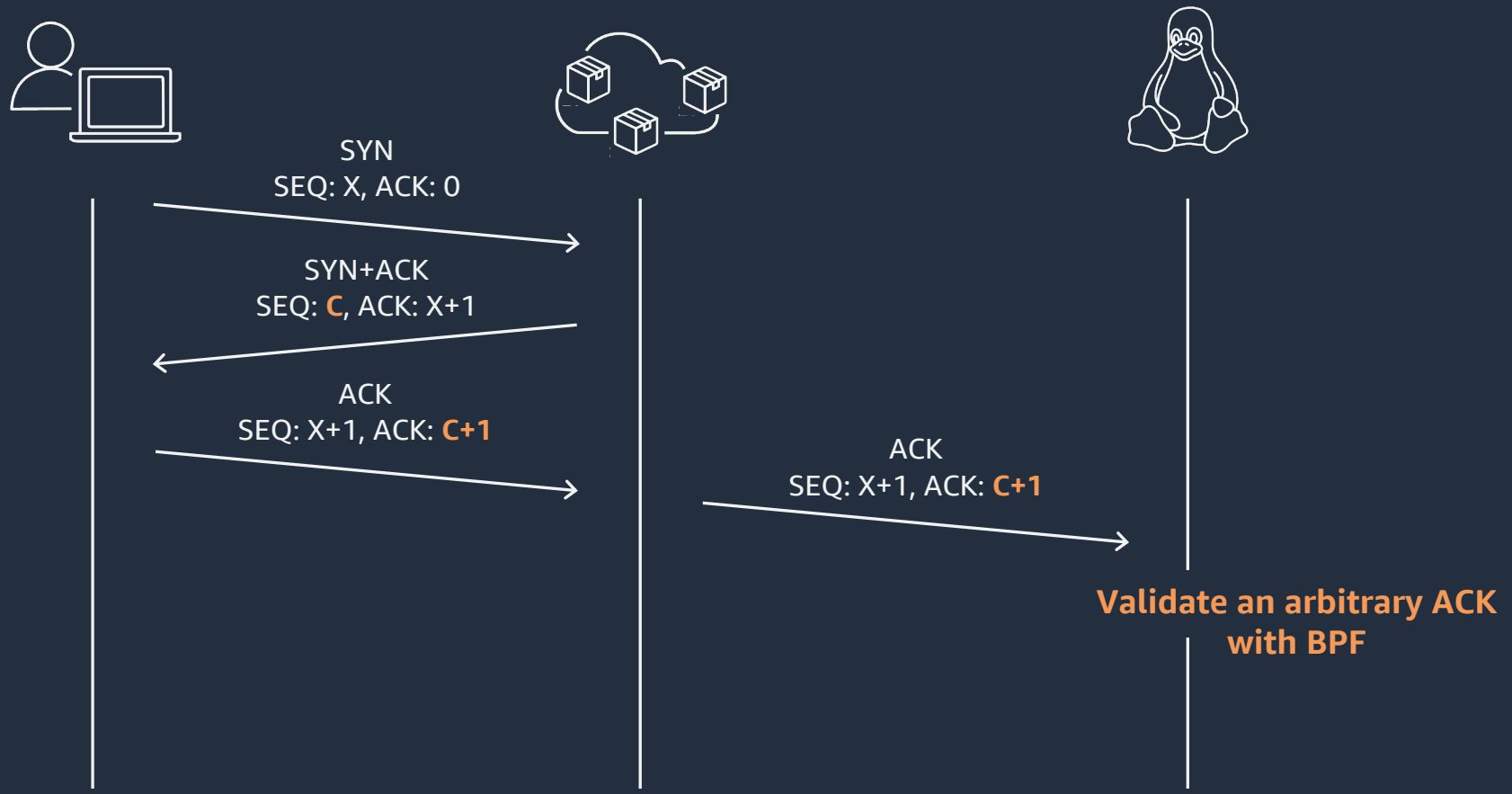
© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

SYN Proxy at Scale with BPF



© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

BPF SYN Cookie validation

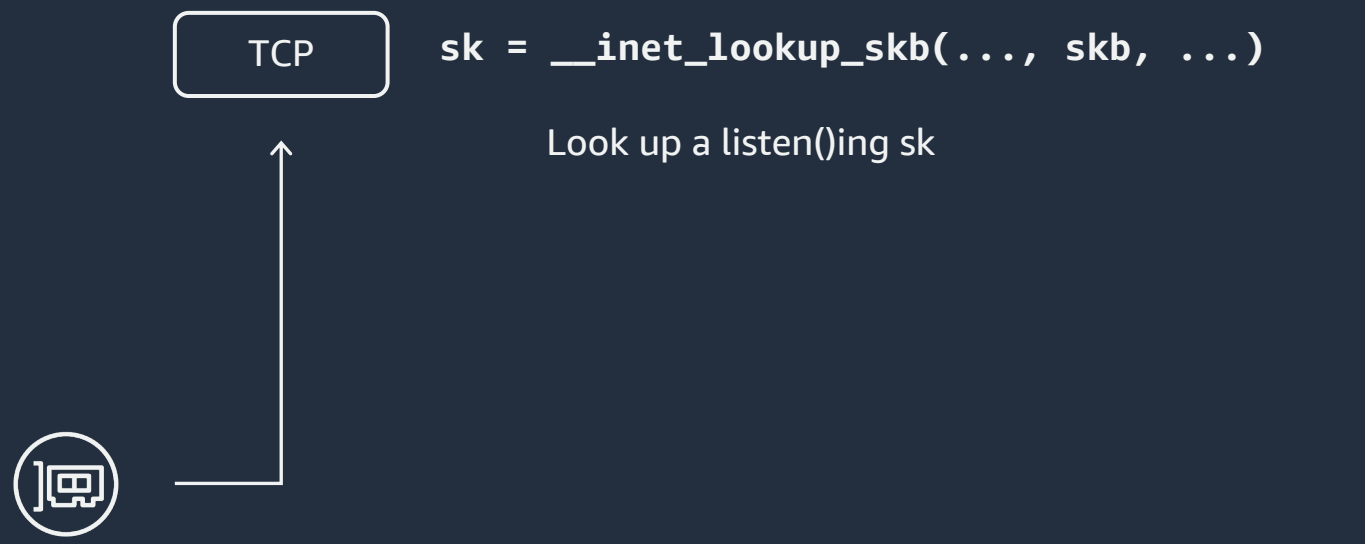


© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Where to invoke BPF ?

BPF prog need to

- Access to TCP header
- Configure reqsk
  - MSS
  - WScale
  - SACK
  - ECN

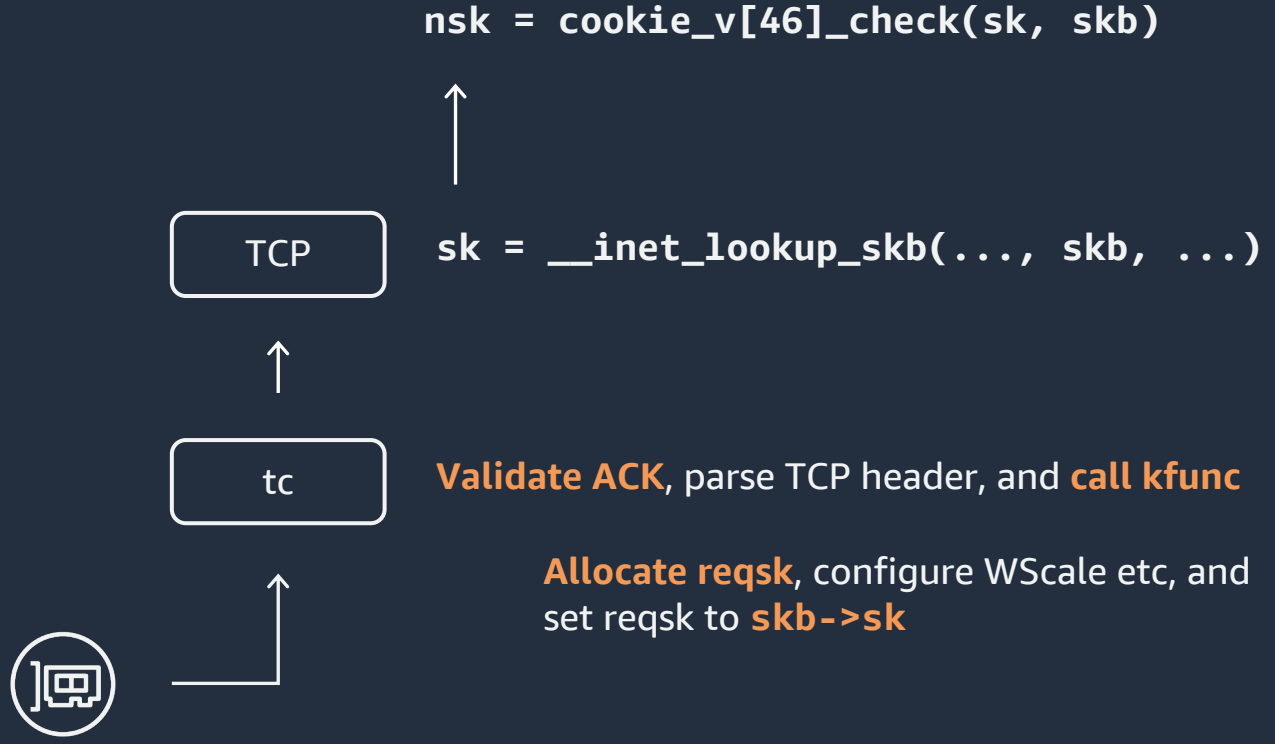


© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Where to invoke BPF ?

BPF prog need to

- Access to TCP header
- Configure reqsk
  - MSS
  - WScale
  - SACK
  - ECN

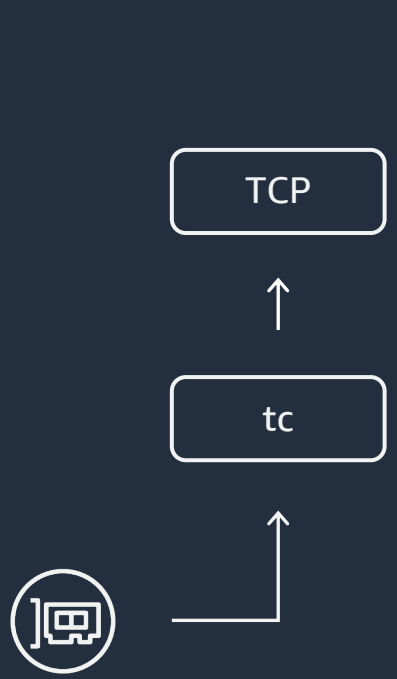


© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Where to invoke BPF ?

BPF prog need to

- Access to TCP header
- Configure reqsk
  - MSS
  - WScale
  - SACK
  - ECN



`nsk = cookie_v[46]_check(sk, skb)`

`sk = __inet_lookup_skb(..., skb, ...)`

Steal `inet_reqsk(skb->sk)->rsk_listener`

Validate ACK, parse TCP header, and **call kfunc**

Allocate reqsk, configure WScale etc, and set reqsk to `skb->sk`

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Where to invoke BPF ?

BPF prog need to

- Access to TCP header
- Configure reqsk
  - MSS
  - WScale
  - SACK
  - ECN



tc

TCP

`nsk = cookie_v[46]_check(sk, skb)`

Skip default validation if `skb->sk != NULL`

`sk = __inet_lookup_skb(..., skb, ...)`

Steal `inet_reqsk(skb->sk)->rsk_listener`

Validate ACK, parse TCP header, and **call kfunc**

Allocate reqsk, configure WScale etc, and set reqsk to `skb->sk`

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

How to call a new kfunc

```
SEC("tc")
int tcp_syncookie(struct __sk_buff *skb)
{
...
        skc = bpf_skc_lookup_tcp(...);                                  /* 1. Dissect ACK. */
        sk = (struct sock *)bpf_skc_to_tcp_sock(skc);                   /* 2. Look up a listener. */
...
        struct tcp_cookie_attributes attr = {                           /* 3. Validate ACK,
                .tcp_opt = {                                             *    parse TCP header,
                        .mss_clamp = mss,                                *    and populate struct
                        .wscale_ok = wscale_ok,                          *    for reqsk configuration
                        .snd_scale = send_scale,                         */
                        .tstamp_ok = tstamp_ok,
                        .sack_ok = sack_ok,
                },
                .ecn_ok = ecn_ok,
        };

        bpf_sk_assign_tcp_reqsk(skb, sk, attr, sizeof(attr));           /* 4. Create a reqsk */

        bpf_sk_release(skc);
        return TC_ACT_OK;
}
```

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

How to call a new kfunc

```
SEC("tc")
int tcp_syncookie(struct __sk_buff *skb)
{
...
        skc = bpf_skc_lookup_tcp(...);                                  /* 1. Dissect ACK. */
        sk = (struct sock *)bpf_skc_to_tcp_sock(skc);                   /* 2. Look up a listener. */
...
        struct tcp_cookie_attributes attr = {                           /* 3. Validate ACK,
                .tcp_opt = {                                             *    parse TCP header,
                        .mss_clamp = mss,                                *    and populate struct
                        .wscale_ok = wscale_ok,                          *    for reqsk configuration
                        .snd_scale = send_scale,                         */
                        .tstamp_ok = tstamp_ok,
                        .sack_ok = sack_ok,
                },
                .ecn_ok = ecn_ok,
        };

        bpf_sk_assign_tcp_reqsk(skb, sk, attr, sizeof(attr));           /* 4. Create a reqsk */

        bpf_sk_release(skc);
        return TC_ACT_OK;
}
```

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

How to call a new kfunc

```
SEC("tc")
int tcp_syncookie(struct __sk_buff *skb)
{
...
        skc = bpf_skc_lookup_tcp(...);                          /* 1. Dissect ACK. */
        sk = (struct sock *)bpf_skc_to_tcp_sock(skc);           /* 2. Look up a listener. */
...
        struct tcp_cookie_attributes attr = {                   /* 3. Validate ACK,
                .tcp_opt = {                                     *    parse TCP header,
                        .mss_clamp = mss,                        *    and populate struct
                        .wscale_ok = wscale_ok,                  *    for reqsk configuration
                        .snd_scale = send_scale,                 */
                        .tstamp_ok = tstamp_ok,
                        .sack_ok = sack_ok,
                },
                .ecn_ok = ecn_ok,
        };

        bpf_sk_assign_tcp_reqsk(skb, sk, attr, sizeof(attr));   /* 4. Create a reqsk */

        bpf_sk_release(skc);
        return TC_ACT_OK;
}
```

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

How to call a new kfunc

```
SEC("tc")
int tcp_syncookie(struct __sk_buff *skb)
{
...
        skc = bpf_skc_lookup_tcp(...);                                  /* 1. Dissect ACK. */
        sk = (struct sock *)bpf_skc_to_tcp_sock(skc);                   /* 2. Look up a listener. */
...
        struct tcp_cookie_attributes attr = {                           /* 3. Validate ACK,
                .tcp_opt = {                                             *    parse TCP header,
                        .mss_clamp = mss,                                *    and populate struct
                        .wscale_ok = wscale_ok,                          *    for reqsk configuration
                        .snd_scale = send_scale,                         */
                        .tstamp_ok = tstamp_ok,
                        .sack_ok = sack_ok,
                },
                .ecn_ok = ecn_ok,
        };

        bpf_sk_assign_tcp_reqsk(skb, sk, attr, sizeof(attr));           /* 4. Create a reqsk */

        bpf_sk_release(skc);
        return TC_ACT_OK;
}
```

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

How to call a new kfunc

```
SEC("tc")
int tcp_syncookie(struct __sk_buff *skb)
{
...
        skc = bpf_skc_lookup_tcp(...);                          /* 1. Dissect ACK. */
        sk = (struct sock *)bpf_skc_to_tcp_sock(skc);           /* 2. Look up a listener. */
...
        struct tcp_cookie_attributes attr = {                   /* 3. Validate ACK,
                .tcp_opt = {                                     *    parse TCP header,
                        .mss_clamp = mss,                        *    and populate struct
                        .wscale_ok = wscale_ok,                  *    for reqsk configuration
                        .snd_scale = send_scale,                 */
                        .tstamp_ok = tstamp_ok,
                        .sack_ok = sack_ok,
                },
                .ecn_ok = ecn_ok,
        };

        bpf_sk_assign_tcp_reqsk(skb, sk, attr, sizeof(attr));   /* 4. Create a reqsk */

        bpf_sk_release(skc);
        return TC_ACT_OK;
}
```

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

struct tcp_cookie_attributes

Include/net/tcp.h

```
struct tcp_cookie_attributes {
        struct tcp_options_received tcp_opt;
        bool ecn_ok;
} __attribute__((__packed__));    /* To extend in the future  e.g. tcp_usec_ts */
```

include/linux/tcp.h

```
struct tcp_options_received {
...
        u16     ...
                tstamp_ok  : 1,  /* TIMESTAMP seen on SYN packet */
                wscale_ok  : 1,  /* Wscale seen on SYN packet */
                sack_ok    : 3,  /* SACK seen on SYN packet */
                snd_wscale : 4,  /* Window scaling received from sender */
                rcv_wscale : 4;  /* Window scaling to send to receiver */
        u16     mss_clamp;       /* Maximal mss, negotiated at connection setup */
};
```

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

bpf_sk_assign_tcp_reqsk

net/core/filter.c

```
__bpf_kfunc int bpf_sk_assign_tcp_reqsk(struct sk_buff *skb, struct sock *sk,
                                        struct tcp_cookie_attributes *attr, int attr__sz)
{
...
        req = inet_reqsk_alloc(ops, sk, false);          /* Not increment listener's refcnt */
        req->syncookie = 1;
        req->rsk_listener = sk;                          /* Need to be cleared before freeing */
        req->mss = attr->tcp_opt.mss_clamp;

        ireq = inet_rsk(req);
        ireq->snd_wscale = attr->tcp_opt.snd_wscale;
        ireq->wscale_ok = attr->tcp_opt.wscale_ok;
...                                                      /* Not fully initialised here.
        skb_orphan(skb);                                  * e.g. snt_isn will be set later.
        skb->sk = req_to_sk(req);                         */
        skb->destructor = sock_pfree;

        return 0;
}
```

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

bpf_sk_assign_tcp_reqsk

net/core/filter.c

```
__bpf_kfunc int bpf_sk_assign_tcp_reqsk(struct sk_buff *skb, struct sock *sk,
                                        struct tcp_cookie_attributes *attr, int attr__sz)
{
...
        req = inet_reqsk_alloc(ops, sk, false);          /* Not increment listener's refcnt */
        req->syncookie = 1;
        req->rsk_listener = sk;                          /* Need to be cleared before freeing */
        req->mss = attr->tcp_opt.mss_clamp;

        ireq = inet_rsk(req);
        ireq->snd_wscale = attr->tcp_opt.snd_wscale;
        ireq->wscale_ok = attr->tcp_opt.wscale_ok;
...                                                      /* Not fully initialised here.
        skb_orphan(skb);                                  * e.g. snt_isn will be set later.
        skb->sk = req_to_sk(req);                         */
        skb->destructor = sock_pfree;

        return 0;
}
```

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

bpf_sk_assign_tcp_reqsk

net/core/filter.c

```
__bpf_kfunc int bpf_sk_assign_tcp_reqsk(struct sk_buff *skb, struct sock *sk,
                                        struct tcp_cookie_attributes *attr, int attr__sz)
{
...
        req = inet_reqsk_alloc(ops, sk, false);           /* Not increment listener's refcnt */
        req->syncookie = 1;
        req->rsk_listener = sk;                           /* Need to be cleared before freeing */
        req->mss = attr->tcp_opt.mss_clamp;

        ireq = inet_rsk(req);
        ireq->snd_wscale = attr->tcp_opt.snd_wscale;
        ireq->wscale_ok = attr->tcp_opt.wscale_ok;
...                                                       /* Not fully initialised here.
        skb_orphan(skb);                                   * e.g. snt_isn will be set later.
        skb->sk = req_to_sk(req);                          */
        skb->destructor = sock_pfree;

        return 0;
}
```

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

sock_pfree

net/core/sock.c

```
void sock_pfree(struct sk_buff *skb)
{
        struct sock *sk = skb->sk;

        if (!sk_is_refcounted(sk))
                return;

        if (sk->sk_state == TCP_NEW_SYN_RECV && inet_reqsk(sk)->syncookie) {
                inet_reqsk(sk)->rsk_listener = NULL;
                reqsk_free(inet_reqsk(sk));
                return;
        }

        sock_gen_put(sk);
}
```

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

inet_steal_sock

include/net/inet_hashtables.h

```
static inline struct sock *inet_steal_sock(struct net *net, struct sk_buff *skb, int doff,
...
{
...
        sk = skb_steal_sock(skb, refcounted, &prefetched);
        if (!sk)
                return NULL;

        if (!prefetched) /* skb->sk != sock_pfree() */
                return sk;

        if (sk->sk_state == TCP_NEW_SYN_RECV) {
                if (inet_reqsk(sk)->syncookie) {
                        *refcounted = false;
                        skb->sk = sk;
                        skb->destructor = sock_pfree;
                        return inet_reqsk(sk)->rsk_listener;
                }
...
```

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

cookie_v[46]_check

include/net/tcp.h

```
#if IS_ENABLED(CONFIG_BPF)
static inline bool cookie_bpf_ok(struct sk_buff *skb)
{
        return skb->sk;
}
...
```

net/ipv4/syncookies.c

```
struct sock *cookie_v4_check(struct sock *sk, struct sk_buff *skb)
{
...
        if (cookie_bpf_ok(skb))
                req = cookie_bpf_check(net, sk, skb);      /* Fully initialised. */
```

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Conclusion

SYN Cookie in Linux

- Statefull and not scalable
- Sacrifice performance if TS is not supported

Custom SYN Cookie with BPF

- Stateless and scalable

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.



Thank you!

Kuniyuki Iwashima
kuniyu@amazon.com

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.